

Base de datos relacional de Vinos



Elegí esta temática porque, al tratarse de un trabajo personal, me permite materializar en un sistema de base de datos relacional la relación entre lo comercial y la información de un rubro en el que estuve inmerso durante años trabajando en una vinoteca.

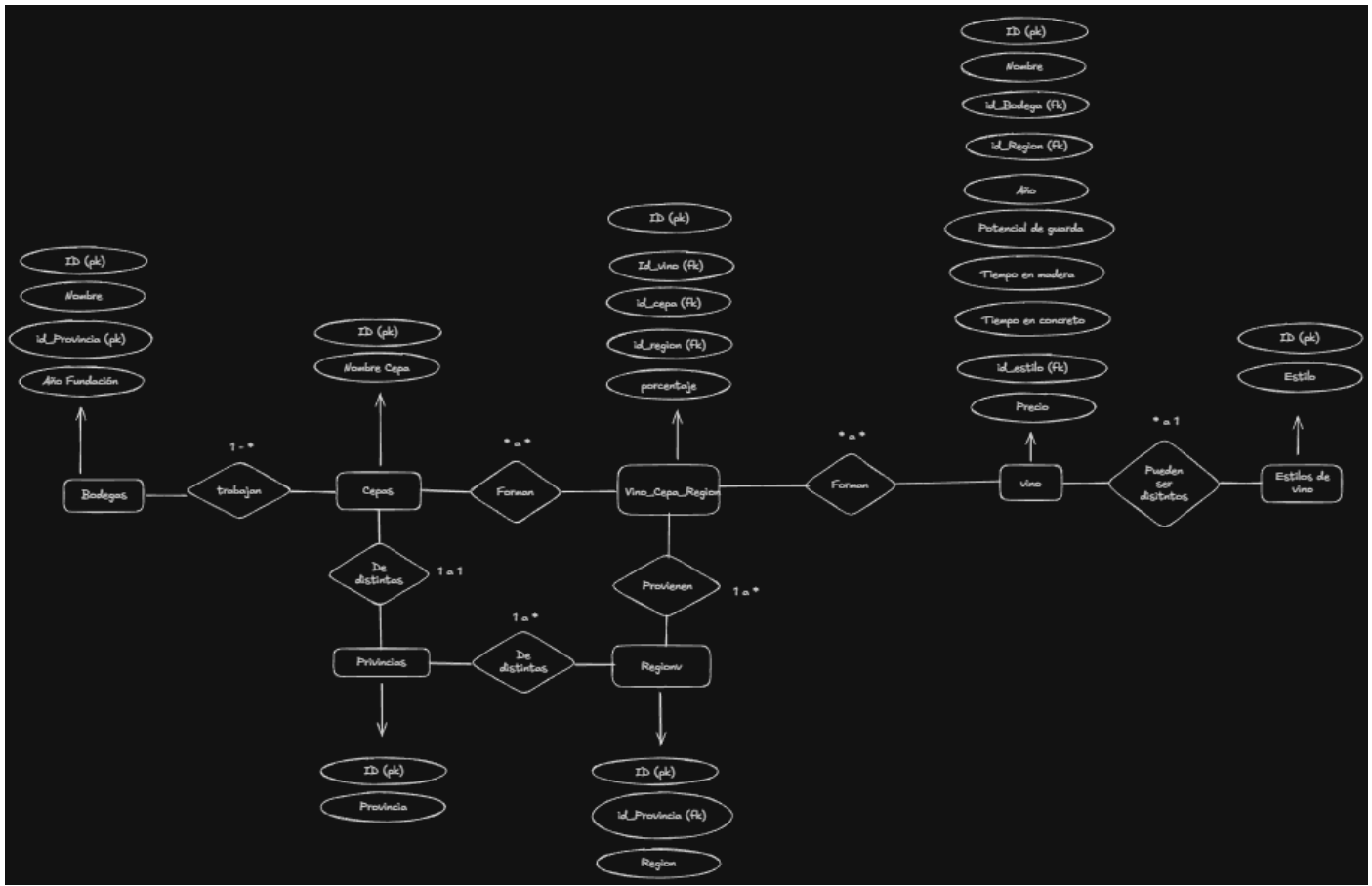


Cada vino puede estar compuesto por una o varias uvas, y puede clasificarse como tinto, rosado, blanco o espumante.

Contar con una base de datos relacional permite organizar la información de manera eficiente, separando qué vino contiene qué uvas, si tuvo paso por madera, por huevos de hormigón, potencial de guarda y el año de cosecha.

Esta estructura es fundamental para una vinoteca, ya que facilita la gestión, consulta y mantenimiento de los datos.

Diagrama de Entidad Relación (D.E.R)



Listado de las tablas que comprenden la base de datos, con descripción de cada tabla, listado de campos, abreviaturas de nombres de campos, nombres completos de campos, tipos de datos, tipo de clave (foránea, primaria, índice(s))

Bodega: Entidad que representa el establecimiento vitivinícola responsable de la elaboración de los vinos.

Campos:

- 1) identificación única por bodega.
- 2) Nombre de la bodega
- 3) Provincia donde está la bodega
- 4) Año de fundación

Campo	Tipo	Clave
ID	INT	PK
Nombre	VARCHAR	INDEX
id_Provincia	INT	FK
Anio_fundacion	INT	INDEX

Cepa: Entidad que representa la variedad de uva utilizada en la elaboración del vino, como, por ejemplo: Malbec, Cabernet Sauvignon, Syrah, Chardonnay, etc.

Campos:

- 1) identificación única por variedad.
- 2) Nombre de la variedad.

Campo	Tipo	Clave
ID	INT	PK
Cepa	VARCHAR	INDEX

Provincia: Entidad que referencia una zona geográfica de donde está plantado el viñedo, lo que da información importante para anticipar las cualidades del vino.

Campos:

- 1) identificación única por provincia.
- 2) Nombre de la provincia.

Campo	Tipo	Clave
ID	INT	PK
Provincia	VARCHAR	INDEX

Región: Entidad que referencia la región dentro de la provincia donde está plantado el viñedo, lo que también da información sobre las características del vino.

Campos:

- 1) identificación única por región.
- 2) Nombre de la región
- 3) Clave foránea que referencia a la provincia a la que pertenece la región.

Campo	Tipo	Clave
ID	INT	PK
Region	VARCHAR	INDEX
id_Provincia	INT	FK

Vino Cepa Region: Entidad asociativa que representa la relación entre los vinos, las cepas y las zonas donde producen las cepas que los componen, permitiendo modelar la participación de una o varias variedades de uva en la elaboración de un vino. Al añadir a la región en esta tabla intermedia se resuelve la

problemática que puede generar tener una misma cepa, pero de distintas zonas en un mismo vino. Incluye el porcentaje de cada cepa utilizada.

Campos:

- 1) identificación única.
- 2) Clave foránea que referencia a la identidad vino.
- 3) Clave foránea que referencia a la identidad cepa.
- 4) Clave foránea que referencia a la identidad región.
- 5) Porcentaje de participación de la cepa en el vino (0 – 100).

Campo	Tipo	Clave
ID	INT	PK
Id_vino	INT	FK
Id_cepa	INT	FK
Id_region	INT	FK
Porcentaje	DECIMAL	---

Vino: Entidad que representa el producto final del proceso vitivinícola, resultado de la combinación de cepas, regiones de origen y métodos de elaboración.

Campos:

- 1) Identidad única.
- 2) Nombre.
- 3) Clave foránea que referencia a la identidad bodega.
- 4) Clave foránea que referencia a la identidad región.
- 5) Año del vino.
- 6) Potencial de tiempo que tiene el vino para ser guardado antes de empezar a perder sus propiedades, expresado en años.
- 7) Tiempo representado en meses que estuvo el vino en barricas de roble.
- 8) Tiempo representado en meses que estuvo el vino en huecos de concreto.
- 9) Clave foránea que referencia a la identidad estilo del vino (blanco, espumante, tinto, dulce).
- 10) Precio de lista.

Campo	Tipo	Clave
Id	INT	PK
Nombre	VARCHAR	INDEX
Id_Bodega	INT	FK
Id_Region	INT	FK
Año	INT	INDEX
Potencial	INT	INDEX
Tiempo_madera	INT	INDEX
Tiempo_concreto	INT	INDEX
Id_Estilo	INT	FK
Precio	DECIMAL	INDEX

Estilos de vino: Entidad que representa los distintos estilos de elaboración del vino, como vinos tintos, blancos, rosados, espumantes y dulces, entre otros.

Campos:

- 1) Identidad única.
- 2) Estilo del vino

Campo	Tipo	Clave
Id	INT	PK
Estilo	VARCHAR	INDEX

- Listado de Vistas más una descripción detallada, su objetivo, y qué tablas las componen.

Vista 1) Filtra vinos de la provincia de mendoza. El objetivo es mostrar al cliente los vinos que se tiene de la zona de mendoza, asegurándonos de no dar información de otros vinos que no sean de esa provincia. Componen la tabla de vino, región y provincia.

```
CREATE VIEW VistaVinosMendocinos AS SELECT
v.id,
v.nombre,
v.precio,
r.region,
p.provincia
FROM vino v
JOIN region r ON v.id_region = r.id
JOIN provincia p ON r.id_provincia = p.id
WHERE p.provincia = "Mendoza"
ORDER BY v.precio ASC;
```

Vista 2) Realiza un promedio de precio del total de la base de datos. El objetivo es tener una referencia en general del publico al que se apunta, saber que segmento de etiquetas ampliar, ya sea para tener mas publico que compra vinos para todos los días, o si se necesita mas vinos premium. Componen la tabla vino

```
CREATE VIEW vistaPrecioPromedio AS
SELECT
COUNT(v.precio) AS CantidadVinos,
ROUND(AVG(v.precio)) AS precioPromedio
FROM vino v;
```

Vista 3) Realiza un filtro de Vinos blend, que son aquellos que son compuestos por mas de una cepa (uva). El objetivo es tener una tabla accesible que simplifique la consulta sobre vinos con esta característica, ya que no pueden ser filtrados por una única cepa. Componen la tabla de vino, bodega y vino_cepa_region.

```
CREATE VIEW vistaVinosBlend AS
SELECT
v.id,
v.nombre AS vino,
v.precio,
b.nombre AS bodega,
COUNT(DISTINCT vcr.id_cepa) AS cantidad_cepas,
GROUP_CONCAT(DISTINCT c.nombre ORDER BY c.nombre SEPARATOR ", ") AS cepas
FROM vino v
JOIN vino_cepa_region vcr ON v.id = vcr.id_vino
JOIN bodega b ON v.id_bodega = b.id
JOIN cepa c ON vcr.id_cepa = c.id
GROUP BY v.id, v.nombre, v.precio, b.nombre
HAVING COUNT(DISTINCT vcr.id_cepa) > 1
ORDER BY v.precio ASC;
```

Vista 4) Realiza un filtro por estilo de vino, en este caso de vinos espumantes. Es una consulta muy útil para cuando el cliente sabe lo que quiere o para analizar los productos disponibles de dicho segmento. Se utilizó la tabla de vino y estilo_de_vino.

```
CREATE VIEW vistaVinosEspumantes AS SELECT
v.nombre,
v.precio,
e.estilo
FROM vino v
JOIN estilo_de_vino e ON v.id_estilo = e.id
WHERE e.estilo = "Espumante"
ORDER BY v.precio ASC;
```

Vista 5) Esta vista ordena los vinos por mayor potencial de guarda. Muchas veces los clientes buscan vinos que tengan cierto potencial de guarda para lograr un vino evolucionado. Se utilizaron las tablas de vino, bodega y región.

```
CREATE VIEW vistaVinosConMasPotencial AS SELECT
v.nombre,
v.precio,
v.potencial,
b.nombre AS bodega,
r.region
FROM vino v
JOIN bodega b ON v.id_bodega = b.id
JOIN region r ON v.id_region = r.id
WHERE v.potencial >= 10
ORDER BY v.potencial DESC;
```

- Listado de Funciones que incluyan una descripción detallada, el objetivo para la cual fueron creadas y qué datos o tablas manipulan y/o son implementadas.

1) Funcion donde la pasamos el id de la bodega y devuelve el promedio de precios de los vinos de esa bodega que tenemos en la base de datos. Se utiliza la tabla de bodega para determinar que bodega estamos consultando y la tabla de vinos para obtener el vino correcto y los precios.

```
DELIMITER //
CREATE FUNCTION fn_precio_promedio_bodega (parametro_bodega INT)
RETURNS DECIMAL(10,2)
DETERMINISTIC
BEGIN
DECLARE valorRetorno DECIMAL(10,2);

SELECT AVG(precio) INTO valorRetorno
FROM vino
WHERE id_bodega = parametro_bodega;

RETURN valorRetorno;
END;
//
DELIMITER ;
```

Función 2) que devuelve el nombre de una bodega a partir de su ID (puede reemplazar un JOIN). Se utiliza la tabla bodega.

```
DELIMITER //
CREATE FUNCTION fn_nombre_bodega(parametro_id INT)
RETURNS VARCHAR(80)
DETERMINISTIC
BEGIN
DECLARE valorRetorno VARCHAR(80);

SELECT b.nombre INTO valorRetorno
FROM bodega b
WHERE b.id = parametro_id;

RETURN valorRetorno;
END; //
DELIMITER ;
```

- Listado de Stored Procedures con una descripción detallada, qué objetivo o beneficio aportan al proyecto, y las tablas que lo componen y/o tablas con las que interactúa.

Stored producer 1) Ordena los vinos, eligiendo que campo de ordenamiento y de que forma (ASC o DESC). Se utiliza para flexibilizar la consulta de vinos.

Los campos pueden ser id, nombre, id_bodega, id_region, id_estilo, anio, potencial, tiempo_madera, tiempo_concreto o precio. Solo se involucra la tabla vino.

```
DELIMITER //
CREATE PROCEDURE sp_ordenar_vinos(
  IN _campo VARCHAR(50),
  IN _orden VARCHAR(4) )
BEGIN

  DECLARE consulta_dinamica TEXT;

  SET @consulta_dinamica = CONCAT('SELECT * FROM vino ORDER BY ', _campo, ' ', _orden);

  PREPARE stmt FROM @consulta_dinamica;
  EXECUTE stmt;
  DEALLOCATE PREPARE stmt;

END
//
DELIMITER ;
```

Stored producer 2) Se utiliza para ingresar un nuevo vino a la tabla vino. Parametros: nombre, id_bodega, id_region, id_estilo, año (de la cosecha), potencial (expresado en años), tiempo medera

(expresado en meses), tiempo concreto (expresado en meses) y precio. El objetivo es centralizar la lógica de inserciones. Tablas involucradas vino, bodega, región, estilo_de_vino

```
DELIMITER //
CREATE PROCEDURE sp_insertar_nuevo_vino (
  IN _nombre VARCHAR(100),
  IN _id_bodega INT,
  IN _id_region INT,
  IN _id_estilo INT,
  IN _anio INT,
  IN _potencial INT,
  IN _tiempo_madera INT,
  IN _tiempo_concreto INT,
  IN _precio DECIMAL(10, 2)
)

BEGIN

  INSERT INTO vino(
    nombre,
    id_bodega,
    id_region,
    id_estilo,
    anio,
    potencial,
    tiempo_madera,
    tiempo_concreto,
    precio
  )

  VALUES(
    _nombre,
    _id_bodega,
    _id_region,
    _id_estilo,
    _anio,
    _potencial,
    _tiempo_madera,
    _tiempo_concreto,
    _precio);

END
//
DELIMITER ;
```

- Se detalla el listado de Triggers creados, una descripción de su funcionalidad, objetivos y/o sobre qué tablas y/o situaciones se accionan.

Trigger 1) Trigger con su tabla de auditoria para cuando haya modificación de precios en la tabla “vino”. Se involucra la tabla vino y la tabla de auditoria_precios.

```
CREATE TABLE auditoria_precios (  
  id INT AUTO_INCREMENT,  
  id_vino INT,  
  precio_anterior DECIMAL(10,2),  
  precio_nuevo DECIMAL(10,2),  
  fecha_cambio DATETIME,  
  PRIMARY KEY (id)  
);  
DELIMITER //  
  
CREATE TRIGGER tr_cambio_precio  
AFTER UPDATE ON vino  
FOR EACH ROW  
BEGIN  
  
  IF OLD.precio <> NEW.precio THEN  
    INSERT INTO auditoria_precios (  
      id_vino,  
      precio_anterior,  
      precio_nuevo,  
      fecha_cambio  
    )  
      VALUES (  
        OLD.id,  
        OLD.precio,  
        NEW.precio,  
        NOW()  
      );  
  END IF;  
END //  
  
DELIMITER ;
```

Trigger 2) Evitar precios inválidos, no permite que se modifique un precio a 0 o menos, se involucra con la tabla vinos, se accionan antes de que se intente modificar el precio.

```
DELIMITER //  
  
CREATE TRIGGER tr_precio_invalido  
BEFORE UPDATE ON vino  
FOR EACH ROW  
BEGIN  
  IF NEW.precio <= 0 THEN  
    SIGNAL SQLSTATE '45000'  
    SET MESSAGE_TEXT = 'El precio debe ser mayor que cero';  
  END IF;  
END  
//  
DELIMITER ;
```